

SoC RISC-V en FPGA con FemtoRV32

Implementación de Calculadora Básica:
Sumadora sobre Canal UART

Universidad Nacional de Colombia
Programa: Ingeniería Electrónica
Asignatura: Arquitectura de Computadores

FPGA: Tang Primer 20K — GW2A-LV18PG256C8/I7
Procesador: FemtoRV32 Quark (RV32I)
Herramienta: Gowin EDA 1.9.12 | Toolchain: xpack-riscv-none-elf-gcc-15.2.0

Junio 2026

Resumen

Este documento describe el diseño, implementación y proceso de depuración de una calculadora básica de suma ejecutada sobre el SoC FemtoUN. El programa constituye la primera etapa de lógica de aplicación construida sobre el canal UART previamente validado mediante el programa de eco ASCII. Se documentan cinco hallazgos identificados durante la depuración, cada uno con su síntoma observable, causa raíz diagnosticada y corrección aplicada o hipótesis activa. Al cierre del presente informe, la operación de suma produce resultados correctos para operandos de un dígito en todos los casos probados; la suma de operandos con dos o más dígitos presenta resultados incorrectos cuya causa raíz permanece bajo investigación.

Repositorio: `femtorv2-soc-fpga` | Licencia MIT | Verilog + C bare-metal

Índice

1. Contexto y Objetivo	2
1.1. Posición en el flujo de desarrollo	2
1.2. Objetivos específicos	2
2. Diseño del Firmware de la Sumadora	2
2.1. Principio de operación	2
2.2. Acumulación de dígitos sin multiplicación nativa	3
2.3. Impresión del resultado sin división nativa	3
2.4. Lazo principal	3
3. Mapa de Registros del Periférico UART	4
4. Parámetros del Sistema	5
5. Hallazgos Durante la Depuración	5
5.1. FIX 1 – Retardo por software insuficiente en <code>leer_char</code>	5
5.2. FIX 2 – Verificación del comportamiento de <code>leer_char</code>	6
5.3. FIX 3 – Inicialización tardía del puntero de pila	6
5.4. Problema abierto – Resultados incorrectos con operandos de dos dígitos	7
6. Resultados y Verificación Funcional	7
6.1. Evidencia de operación	7
6.2. Análisis de la salida	8
6.3. Resumen de pruebas	8
7. Conclusiones	8
A. Resumen de Correcciones Aplicadas	9
B. Comando de Compilación Utilizado	9
C. Código Fuente Final	10

1 Contexto y Objetivo

1.1 Posición en el flujo de desarrollo

El presente informe es continuación directa del trabajo documentado en *Verificación del Canal UART mediante Programa de Eco ASCII*, que estableció las condiciones operativas del subsistema de comunicación serial del SoC FemtoUN. Una vez confirmada la integridad bidireccional del canal, el paso siguiente en el flujo de desarrollo consiste en construir lógica de aplicación que haga uso de dicho canal: la calculadora básica.

La calculadora implementa una máquina de estados simple que solicita al usuario dos operandos numéricos y un operador aritmético a través de la terminal PuTTY, ejecuta la operación sobre el procesador FemtoRV32 Quark y devuelve el resultado por el mismo canal. El presente informe cubre únicamente la operación de suma; las operaciones de resta, multiplicación y división se documentarán en informes posteriores una vez resueltos los problemas identificados en esta etapa.

1.2 Objetivos específicos

1. Implementar la lectura de operandos numéricos de múltiples dígitos desde la terminal UART, acumulando el valor mediante aritmética de desplazamiento de bits.
2. Ejecutar la operación de suma sobre el procesador RV32I y transmitir el resultado formateado hacia la terminal.
3. Identificar y documentar los problemas encontrados durante el proceso de depuración, estableciendo las condiciones necesarias para la implementación de las operaciones aritméticas restantes.

2 Diseño del Firmware de la Sumadora

2.1 Principio de operación

El firmware implementa un lazo continuo con tres fases secuenciales. En la primera fase se captura el primer operando dígito a dígito hasta recibir el carácter de retorno de carro. En la segunda fase se espera el carácter operador, en este caso únicamente el símbolo +. En la tercera fase se captura el segundo operando con el mismo mecanismo. Una vez completadas las tres fases, se calcula la suma y se imprime el resultado en la terminal, tras lo cual el lazo reinicia.

El acceso al periférico UART se realiza mediante macros de preprocesador, siguiendo el principio establecido en el programa de eco ASCII: las direcciones de hardware deben resolverse en tiempo de compilación, sin dependencia de ningún mecanismo de inicialización de sección `.data`.

```
1 #define uart_data ((volatile int *)0x00400008)
2 #define uart_ctrl ((volatile int *)0x00400010)
```

Listing 1: Definición de acceso a registros UART mediante macros.

2.2 Acumulación de dígitos sin multiplicación nativa

El procesador FemtoRV32 Quark implementa el conjunto base RV32I, que no incluye la extensión de multiplicación de hardware. El uso del operador `*` en el código fuente genera llamadas a funciones de biblioteca en tiempo de ejecución (`__mulsi3`) que no están disponibles en un entorno compilado con `-nostartfiles` sin soporte de `libgcc`.

La acumulación del valor numérico a partir de dígitos ASCII se implementa mediante una combinación de desplazamientos de bits equivalente a la multiplicación por diez:

```
1 num1 = (num1 << 3) + (num1 << 1) + (t - '0');
2 /* equivalente a: num1 = num1 * 8 + num1 * 2 + digito
3                  = num1 * 10 + digito          */
```

Listing 2: Acumulación de dígito sin operador de multiplicación.

Esta expresión es algebraicamente correcta y genera únicamente instrucciones de desplazamiento y suma presentes en RV32I, sin llamadas a funciones de runtime.

2.3 Impresión del resultado sin división nativa

Por la misma razón que impide el uso del operador `*`, los operadores `/` y `%` tampoco están disponibles directamente. La conversión del resultado entero a su representación decimal se realiza mediante restas sucesivas para cada potencia de diez, con un rango que cubre hasta 10^9 .

```
1 void imprimir_numero(int n) {
2     if (n == 0) { enviar_char('0'); return; }
3     int d; int iniciado = 0;
4     if (n < 0) { enviar_char('-'); n = -n; }
5
6     d=0; while(n>=1000000000){n-=1000000000;d++;}
7     if(d||iniciado){enviar_char('0'+d);iniciado=1;}
8     /* ... continua para cada potencia de diez ... */
9     d=0; while(n>=1){n-=1;d++;} enviar_char('0'+d);
10 }
```

Listing 3: Fragmento de `imprimir_numero` mediante restas sucesivas.

2.4 Lazo principal

```
1 int main(void) {
2     for (volatile int i = 0; i < 500000; i++); /* retardo inicial */
3
4     while (1) {
5         int num1 = 0, num2 = 0;
6
7         imprimir_str("\r\n=== SUMADORA BASICA ===\r\n");
8
9         imprimir_str("Numero 1 y Enter: ");
10        while (1) {
11            char t = leer_char();
12            if (t >= '0' && t <= '9') {
13                num1 = (num1 << 3) + (num1 << 1) + (t - '0');
14                enviar_char(t);
15            } else if (t == '\r' || t == '\n') break;
```

```

16     }
17
18     imprimir_str("\r\nOperador (+): ");
19     while (1) {
20         char t = leer_char();
21         if (t == '+') { enviar_char(t); break; }
22     }
23
24     imprimir_str("\r\nNumero 2 y Enter: ");
25     while (1) {
26         char t = leer_char();
27         if (t >= '0' && t <= '9') {
28             num2 = (num2 << 3) + (num2 << 1) + (t - '0');
29             enviar_char(t);
30         } else if (t == '\r' || t == '\n') break;
31     }
32
33     imprimir_str("\r\nResultado: ");
34     imprimir_numero(num1 + num2);
35     imprimir_str("\r\n-----\r\n");
36 }
37 return 0;
38 }

```

Listing 4: Estructura del lazo principal de la sumadora.

3 Mapa de Registros del Periférico UART

Los registros del periférico UART y sus campos de bits son idénticos a los documentados en el informe de eco ASCII. Se reproduce aquí únicamente la tabla de referencia rápida para facilitar la lectura del presente documento.

Cuadro 1: Registros del periférico UART accedidos por el firmware de la sumadora.

Dirección	Nombre	Acceso	Descripción
0x00400008	UART_DATA	R/W	Escritura: byte a transmitir. Lectura: byte recibido.
0x00400010	UART_CTRL	R/W	Ver Tabla 2.

Cuadro 2: Campos de bits del registro UART_CTRL (0x00400010).

Bit	Nombre	Descripción
0	tx_wr	Pulso activo alto: inicia transmisión.
1	rx_ack	Pulso activo alto: confirma lectura y limpia rx_avail.
2	LED	Control del LED de usuario.
7	rx_error	Error de trama.
8	rx_avail	Byte recibido disponible para lectura.
9	tx_busy	Transmisor ocupado procesando un envío.

Cuadro 3: Parámetros de hardware y software del entorno de verificación.

Parámetro	Valor
FPGA	Tang Primer 20K — GW2A-LV18PG256C8/I7
Frecuencia de reloj	27 MHz
Baud rate UART	120 000 bps
Duración de trama	$\approx 83,3 \mu\text{s}$ (10 bits a 120 kbps)
Ciclos por trama	$27,000,000 / 120,000 \times 10 = 2250$ ciclos
Memoria BRAM	8192 palabras de 32 bits (32 KiB)
Compilador	xpack-riscv-none-elf-gcc 15.2.0
Flags	-march=rv32i -mabi=ilp32 -Os -nostartfiles
Terminal	PuTTY — COM9, 120000 baud, ISO-8859-1

4 Parámetros del Sistema

5 Hallazgos Durante la Depuración

Durante el proceso de implementación de la sumadora se identificaron cinco problemas. Los tres primeros fueron resueltos exitosamente; los dos últimos corresponden a la causa raíz del comportamiento incorrecto observado con operandos de dos dígitos, que permanece bajo investigación al cierre de este informe.

5.1 FIX 1 – Retardo por software insuficiente en leer_char

Síntoma: Al introducir un número de dos dígitos, la terminal mostraba el mismo dígito impreso dos veces consecutivas y el resultado de la suma era incorrecto. El problema no se manifestaba con operandos de un solo dígito.

Causa: La función `leer_char` utilizaba un retardo por software (`for` con 50 000 iteraciones) para esperar que la bandera `rx_avail` bajara después del pulso `rx_ack`. Con el flag de optimización `-Os`, el compilador reduce significativamente el número de ciclos efectivos del retardo. Al escribir dígitos consecutivos con rapidez normal, el retardo expiraba antes de que `rx_avail` hubiera bajado, y el firmware procesaba el mismo byte por segunda vez.

Corrección: Reemplazar el retardo fijo por la espera activa sobre la bandera, siguiendo el protocolo documentado en el informe de eco ASCII:

```

1 char leer_char(void) {
2     while (!(*uart_ctrl & (1 << 8))); /* esperar rx_avail = 1 */
3     char t = (char)(*uart_data & 0xFF);
4     *uart_ctrl = 2; /* rx_ack = 1 */
5     *uart_ctrl = 0; /* rx_ack = 0 */
6     while (*uart_ctrl & (1 << 8)); /* esperar rx_avail = 0 */
7     return t;
8 }

```

Listing 5: Protocolo correcto de recepción con espera de `rx_avail`.

5.2 FIX 2 – Verificación del comportamiento de leer_char

Síntoma: Tras aplicar FIX 1, los resultados continuaban siendo incorrectos para operandos de dos dígitos. No estaba claro si el problema residía en la función de lectura o en la lógica de acumulación.

Método de diagnóstico: Se implementó un programa de prueba que evaluó tres variantes de `leer_char` en secuencia, imprimiendo el código ASCII de cada byte recibido junto con la variante que lo capturó. El usuario presionó una única tecla por iteración y esperó la respuesta del sistema antes de continuar.

Cuadro 4: Resultados de la prueba de variantes de `leer_char`.

Variante	Mecanismo	Teclas presionadas	Resultado
A	<code>while(rx_avail)</code> tras <code>rx_ack</code>	1, 2, 3	Un byte por tecla
B	Retardo 5 000 iteraciones	4, 5, 6	Un byte por tecla
C	Retardo 10 000 + limpieza	7, 8, 9	Un byte por tecla

Conclusión: Las tres variantes producen exactamente un byte por tecla presionada cuando el usuario introduce las entradas de forma controlada. El problema de resultados incorrectos no se origina en la función de lectura individual, sino en la lógica de acumulación o en el estado del stack durante la ejecución del lazo completo.

5.3 FIX 3 – Inicialización tardía del puntero de pila

Síntoma: Con operandos de un dígito la suma producía resultados correctos en la mayoría de los casos. Con operandos de dos dígitos los resultados eran sistemáticamente incorrectos, con un error que aumentaba proporcionalmente al número de dígitos del primer operando.

Causa: La instrucción `asm volatile("li sp, 0x00002000")` estaba ubicada dentro de `main`. En un flujo compilado con `-nostartfiles`, el código de arranque que normalmente inicializa el puntero de pila antes de la primera instrucción de `main` no existe. El compilador puede generar un prólogo de función para `main` que use el stack antes de que la instrucción de inicialización se ejecute, produciendo escrituras en direcciones arbitrarias.

Corrección: Separar la inicialización del puntero de pila en un archivo `start.S` que se ejecuta antes de transferir el control a `main`:

```

1 .section .text
2 .global _inicio
3 _inicio:
4     li sp, 0x00007FF0      # inicializar stack al final de la BRAM
5     j main                 # saltar al programa principal
6 loop:
7     j loop                 # atrapar retorno inesperado de main

```

Listing 6: Archivo `start.S` con inicialización del puntero de pila.

El comando de compilación se modifica para incluir el archivo de arranque y especificar el punto de entrada:

```

1 riscv-none-elf-gcc -march=rv32i -mabi=ilp32 -Os -nostartfiles -
  nostdlib ^
2 -lgcc "-Wl,-Ttext=0x00000000" "-Wl,-e_inicio" ^
3 -o firmware.elf start.S main.c
4
5 riscv-none-elf-objcopy -O verilog --verilog-data-width=4 ^
6 firmware.elf firmware.hex

```

Listing 7: Comando de compilación con `start.S`.

5.4 Problema abierto – Resultados incorrectos con operandos de dos dígitos

Síntoma: Tras aplicar los tres correctivos anteriores, la suma de operandos de un dígito produce resultados correctos en todos los casos probados. Sin embargo, en cuanto uno de los operandos tiene dos o más dígitos, el resultado es incorrecto. El error no es aleatorio: se observa un desplazamiento sistemático de -7 por cada dígito adicional en el primer operando.

Cuadro 5: Ejemplos de resultados incorrectos observados con operandos de dos dígitos.

Num 1	Num 2	Esperado	Obtenido	Error
10	1	11	4	-7
10	2	12	5	-7
20	1	21	7	-14

Hipótesis principal: La expresión de acumulación $(\text{num1} \ll 3) + (\text{num1} \ll 1) + (t - '0')$ es algebraicamente correcta, pero con `-nostdlib` y `-lgcc` el linker puede estar reubicando la función `_inicio` fuera de la dirección `0x00000000`, de modo que el procesador ejecuta código incorrecto en los primeros ciclos y el stack queda en un estado inconsistente antes de que `_inicio` tome el control. El comando de diagnóstico recomendado es:

```

1 riscv-none-elf-objdump -d firmware.elf | head -30
2 riscv-none-elf-nm firmware.elf | sort

```

Listing 8: Verificación de la posición de `_inicio` en el binario.

Si `_inicio` no aparece en `0x00000000`, la solución es agregar un linker script explícito que fuerce la sección `.text` a comenzar con `_inicio`.

6 Resultados y Verificación Funcional

6.1 Evidencia de operación

La figura 1 muestra la salida del programa de la sumadora en la terminal PuTTY durante una sesión de prueba. La captura incluye tanto operaciones que producen resultados correctos como casos en los que el resultado es incorrecto, lo que permite analizar el patrón de fallo directamente sobre la evidencia experimental.


```
=== SUMADORA BASICA ===
Numero 1 y Enter: 2
Operador (+): +
Numero 2 y Enter: 2
Resultado: 4
-----

=== SUMADORA BASICA ===
Numero 1 y Enter: 9
Operador (+): +
Numero 2 y Enter: 1
Resultado: 10
-----

=== SUMADORA BASICA ===
Numero 1 y Enter: 4
Operador (+): +
Numero 2 y Enter: 6
Resultado: 10
-----

=== SUMADORA BASICA ===
Numero 1 y Enter: 10
Operador (+): +
Numero 2 y Enter: 1
Resultado: 4
-----
```

Figura 1: Salida del programa de sumadora en PuTTY. Se observan resultados correctos para operandos de un dígito y resultados incorrectos cuando el primer operando tiene dos dígitos.

6.2 Análisis de la salida

La frontera entre comportamiento correcto e incorrecto coincide exactamente con la cantidad de dígitos del primer operando: un dígito produce resultados válidos en todos los casos; dos dígitos produce un resultado desplazado hacia abajo en una cantidad proporcional al número de dígitos procesados.

6.3 Resumen de pruebas

7 Conclusiones

1. **La lógica de transmisión y recepción byte a byte opera correctamente.** Los protocolos de acceso al periférico UART establecidos en el programa de eco ASCII se transfieren sin modificación al firmware de la calculadora y producen el comportamiento esperado.
2. **La acumulación de dígitos mediante desplazamientos de bits es algebraicamente correcta.** La expresión $(n \ll 3) + (n \ll 1)$ es equivalente a $n * 10$ para cualquier entero de 32 bits en el rango de trabajo. El problema no reside en esta expresión.
3. **El retardo por software no es un mecanismo confiable para esperar**

Cuadro 6: Contraste entre resultados esperados y obtenidos en la sesión de prueba.

Num 1	Num 2	Esperado	Obtenido	Correcto
2	2	4	4	
9	1	10	10	
4	6	10	10	
10	1	11	4	×

Cuadro 7: Resultados de las pruebas de verificación funcional de la sumadora.

Prueba	Estado	Observación
TX: menú y prompts en terminal		Texto recibido correctamente
RX: lectura de un dígito		Acumulación correcta
Suma de operandos de un dígito		Todos los casos probados
RX: lectura de dos dígitos	×	Resultado con error sistemático
Suma de operandos de dos dígitos	×	Error proporcional a dígitos
Suma de operandos de diferente largo	×	Error en primer operando mayor

`rx_avail`. La optimización `-Os` reduce el número de ciclos efectivos del retardo de forma impredecible. La espera activa sobre la bandera es el único mecanismo que garantiza correctitud independientemente del nivel de optimización del compilador.

4. **La inicialización del puntero de pila debe preceder a cualquier uso de la pila.** La instrucción `li sp` dentro de `main` es insuficiente cuando el compilador genera un prólogo de función que usa el stack antes de ejecutar la primera instrucción del cuerpo de `main`. El uso de `start.S` resuelve este problema estructuralmente.
5. **El problema de resultados incorrectos con operandos de dos dígitos permanece abierto.** La hipótesis más probable es la ubicación incorrecta de `_inicio` fuera de `0x00000000` al compilar con `-nostdlib`lgcc`. La verificación mediante `objdump` y la introducción de un linker script explícito constituyen el siguiente paso de depuración.

A Resumen de Correcciones Aplicadas

B Comando de Compilación Utilizado

```

1 cd "C:\Gowin\Gowin_V1.9.12.01_x64\IDE\bin\Documents\Suma_UNFemto\src"
2
3 riscv-none-elf-gcc -march=rv32i -mabi=ilp32 -Os -nostartfiles -
  nostdlib ^
4 -lgcc "-Wl,-Ttext=0x00000000" "-Wl,-e_inicio" ^
5 -o firmware.elf start.S main.c
6
7 riscv-none-elf-objcopy -O verilog --verilog-data-width=4 ^
8 firmware.elf firmware.hex
9
10 riscv-none-elf-size firmware.elf

```

Listing 9: Comandos de compilación y generación del archivo hex (PowerShell).

Cuadro 8: Resumen de los hallazgos documentados durante la implementación.

FIX	Componente	Problema	Corrección
1	leer_char	Retardo fijo insuficiente con -Os; doble lectura del mismo byte	Espera activa <code>while(rx_avail)</code> tras <code>rx_ack</code>
2	leer_char	Incertidumbre sobre si el problema era de lectura o acumulación	Programa de diagnóstico con tres variantes; confirmado un byte por tecla
3	main.c	Inicialización de <code>sp</code> dentro de <code>main</code> llega tarde	Separar en <code>start.S</code> ; compilar con <code>-e_inicio</code>
–	main.c / start.S	Resultados incorrectos con operandos de dos dígitos (error -7 por dígito)	Problema abierto; hipótesis: <code>_inicio</code> fuera de 0x0

C Código Fuente Final

```

1 #define uart_data ((volatile int *)0x00400008)
2 #define uart_ctrl ((volatile int *)0x00400010)
3
4 void enviar_char(char c);
5 void imprimir_str(const char *s);
6 char leer_char(void);
7 void imprimir_numero(int n);
8
9 int main(void) {
10     for (volatile int i = 0; i < 500000; i++);
11
12     while (1) {
13         imprimir_str("\r\n=== SUMADORA BASICA ===\r\n");
14         int num1 = 0, num2 = 0;
15
16         imprimir_str("Numero 1 y Enter: ");
17         while (1) {
18             char t = leer_char();
19             if (t >= '0' && t <= '9') {
20                 num1 = (num1 << 3) + (num1 << 1) + (t - '0');
21                 enviar_char(t);
22             } else if (t == '\r' || t == '\n') break;
23         }
24
25         imprimir_str("\r\nOperador (+): ");
26         while (1) {
27             char t = leer_char();
28             if (t == '+') { enviar_char(t); break; }
29         }
30
31         imprimir_str("\r\nNumero 2 y Enter: ");
32         while (1) {
33             char t = leer_char();
34             if (t >= '0' && t <= '9') {

```

```

35         num2 = (num2 << 3) + (num2 << 1) + (t - '0');
36         enviar_char(t);
37     } else if (t == '\r' || t == '\n') break;
38 }
39
40     imprimir_str("\r\nResultado: ");
41     imprimir_numero(num1 + num2);
42     imprimir_str("\r\n-----\r\n");
43 }
44 return 0;
45 }
46
47 char leer_char(void) {
48     while (!(*uart_ctrl & (1 << 8)));
49     char t = (char)(*uart_data & 0xFF);
50     *uart_ctrl = 2;
51     *uart_ctrl = 0;
52     while (*uart_ctrl & (1 << 8));
53     return t;
54 }
55
56 void enviar_char(char c) {
57     while (*uart_ctrl & (1 << 9));
58     *uart_data = (int)c;
59     *uart_ctrl = 1;
60     *uart_ctrl = 0;
61 }
62
63 void imprimir_str(const char *s) {
64     while (*s) { enviar_char(*s++); }
65 }
66
67 void imprimir_numero(int n) {
68     if (n == 0) { enviar_char('0'); return; }
69     int d; int iniciado = 0;
70     if (n < 0) { enviar_char('-'); n = -n; }
71
72     d=0; while(n>=1000000000){n-=1000000000;d++;}
73     if(d||iniciado){enviar_char('0'+d);iniciado=1;}
74     d=0; while(n>=100000000){n-=100000000;d++;}
75     if(d||iniciado){enviar_char('0'+d);iniciado=1;}
76     d=0; while(n>=10000000){n-=10000000;d++;}
77     if(d||iniciado){enviar_char('0'+d);iniciado=1;}
78     d=0; while(n>=1000000){n-=1000000;d++;}
79     if(d||iniciado){enviar_char('0'+d);iniciado=1;}
80     d=0; while(n>=100000){n-=100000;d++;}
81     if(d||iniciado){enviar_char('0'+d);iniciado=1;}
82     d=0; while(n>=10000){n-=10000;d++;}
83     if(d||iniciado){enviar_char('0'+d);iniciado=1;}
84     d=0; while(n>=1000){n-=1000;d++;}
85     if(d||iniciado){enviar_char('0'+d);iniciado=1;}
86     d=0; while(n>=100){n-=100;d++;}
87     if(d||iniciado){enviar_char('0'+d);iniciado=1;}
88     d=0; while(n>=10){n-=10;d++;}
89     if(d||iniciado){enviar_char('0'+d);iniciado=1;}
90     d=0; while(n>=1){n-=1;d++;} enviar_char('0'+d);
91 }

```

Listing 10: Código completo de `main.c` – estado al cierre del informe.